

Sprawozdanie z Laboratorium nr 5

Autor: Tomasz Niedziałek

Data: 24.06.2025

1. Wstęp

Celem ćwiczenia było zaimplementowanie algorytmu propagacji wstecznej (ang. backpropagation) dla dwuwarstwowej sieci neuronowej oraz zbadanie jego działania w kontekście konkretnego problemu klasyfikacji. Problem polegał na określeniu, czy dwie liczby rzeczywiste z przedziału $[-1, 1]$ mają ten sam znak.

Zadanie wymagało przetestowania implementacji dla różnych konfiguracji, uwzględniając:

- Normalizację danych wejściowych: dane surowe (nieznormalizowane), dane znormalizowane normą L1 oraz normą L2.
- Funkcje aktywacji w warstwie ukrytej: Sigmoid (oznaczana jako σ) oraz ReLU.
- Różne wartości współczynnika uczenia (LR): w celu zbadania jego wpływu na szybkość i stabilność procesu uczenia.

Niniejsze sprawozdanie przedstawia opis implementacji, analizę uzyskanych wyników oraz wnioski końcowe.

2. Opis Implementacji

2.1. Architektura Sieci

Zgodnie z treścią zadania, zaimplementowano dwuwarstwową sieć neuronową o następującej architekturze:

- Warstwa wejściowa: 2 neurony (odpowiadające dwóm liczbom rzeczywistym).
- Warstwa ukryta: 4 neurony (bramki).
- Warstwa wyjściowa: 1 neuron, zwracający wartość z przedziału $(0, 1)$, która jest interpretowana jako prawdopodobieństwo przynależności do klasy 1 (ten sam znak).

2.2. Funkcje Aktywacji i Funkcja Straty

W warstwie ukrytej testowano dwie funkcje aktywacji:

1. Sigmoid: $\sigma(x) = 1 / (1 + \exp(-x))$
2. ReLU (Rectified Linear Unit): $\text{ReLU}(x) = \max(0, x)$

W warstwie wyjściowej na stałe zastosowano funkcję Sigmoid, co jest standardową praktyką w problemach klasyfikacji binarnej, aby wynik sieci był interpretowalny jako prawdopodobieństwo.

Jako funkcję straty (kosztu) wykorzystano **średni błąd kwadratowy (MSE - Mean Squared Error)**, zdefiniowany jako:

$$\text{MSE} = \text{mean}((y_{\text{pred}} - y_{\text{true}})^2)$$

2.3. Przygotowanie Danych

Dane wejściowe X były generowane losowo jako pary liczb z przedziału $[-1, 1]$. Etykieta y była przypisywana zgodnie z regułą:

- $y = 1$, jeśli $x_1 * x_2 > 0$ (ten sam znak)
- $y = 0$, w przeciwnym wypadku.

Dane były następnie przetwarzane na trzy sposoby:

1. Nieznormalizowane: Dane surowe, bez modyfikacji.
2. Normalizacja L1: Każdy wektor wejściowy był dzielony przez sumę wartości bezwzględnych jego składowych.
3. Normalizacja L2: Każdy wektor wejściowy był dzielony przez jego długość euklidesową.

2.4. Proces Uczenia

Uczenie sieci odbywało się z wykorzystaniem algorytmu propagacji wstecznej i metody spadku gradientu (gradient descent). W każdej z 20,000 epok:

1. Obliczano wyjście sieci dla całego zbioru danych (propagacja w przód).
2. Obliczano błąd (stratę) MSE.
3. Obliczano gradienty wag i biasów dla obu warstw (propagacja wsteczna).
4. Aktualizowano wagi i biasy, odejmując od nich gradienty pomnożone przez współczynnik uczenia.

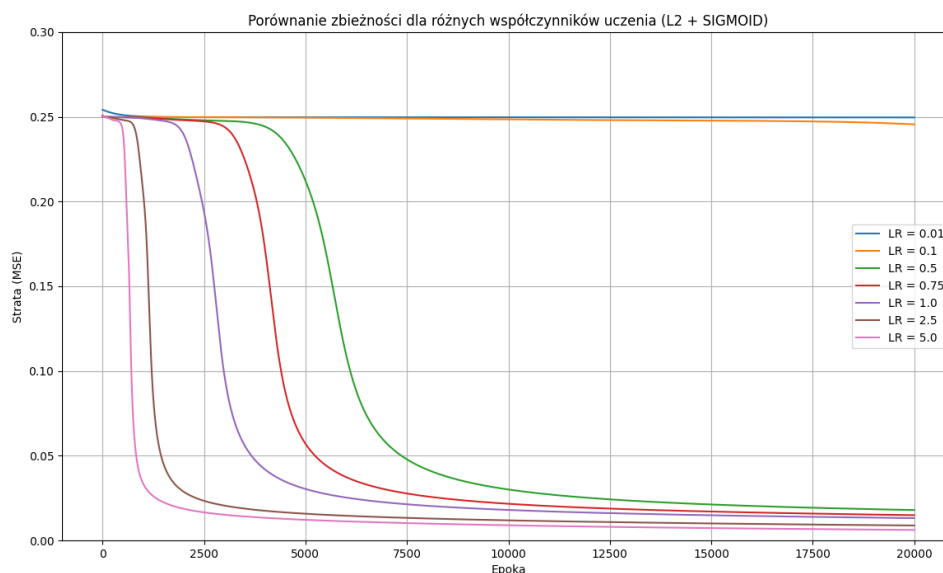
3. Analiza Wyników

Do analizy wykorzystano wykresy zbieżności funkcji straty w zależności od epoki dla różnych konfiguracji.

3.1. Wpływ współczynnika uczenia (LR) na zbieżność

Aby zbadać wpływ samego współczynnika uczenia, wyizolowano jedną, stałą konfigurację sieci (**Normalizacja L2 + aktywacja SIGMOID**) i przeprowadzono trening dla różnych wartości LR.

Wyniki przedstawia poniższy wykres.



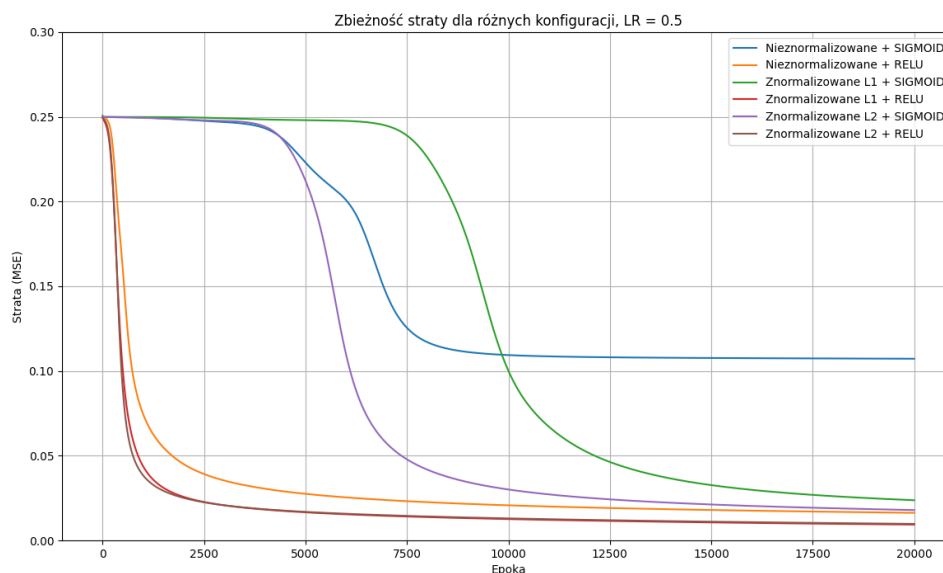
Wykres 1: Porównanie zbieżności funkcji straty dla różnych współczynników uczenia (LR) przy stałej konfiguracji sieci (Normalizacja L2, aktywacja Sigmoid).

Na podstawie Wykresu 1 można wyciągnąć następujące wnioski:

- Zbyt mały LR (0.01, 0.1): Proces uczenia jest bardzo powolny. Strata maleje, ale nawet po 20,000 epok sieć nie osiąga optymalnego wyniku. Gradienty są tak małe, że kroki w kierunku minimum są nieznaczne.
- Optymalny LR (0.5, 0.75, 1.0): W tym zakresie obserwujemy znacznie szybszą zbieżność. Sieci szybko osiągają niski poziom straty i stabilizują się. LR = 1.0 wydaje się zapewniać najlepszy kompromis między szybkością a stabilnością, osiągając najniższą stratę w najkrótszym czasie.
- Zbyt duży LR (2.5, 5.0): Początkowo strata spada bardzo gwałtownie, co może wydawać się pożądane. Jednak szybko stabilizuje się na znacznie wyższym poziomie błędu niż dla optymalnych LR. Dzieje się tak, ponieważ algorytm "przeskakuje" nad minimum lokalnym funkcji straty, a duże kroki uniemożliwiają mu precyzyjne "wcelowanie" w dno doliny błędu. W przypadku LR = 5.0, ostateczna strata jest wyraźnie wyższa niż dla LR = 2.5, co potwierdza niestabilność procesu.

3.2. Wpływ normalizacji danych i funkcji aktywacji

Analizując poszczególne wykresy dla stałych wartości LR (np. LR = 0.5), można ocenić wpływ normalizacji i funkcji aktywacji.



Wykres 2: Porównanie zbieżności dla wszystkich 6 konfiguracji przy $LR = 0.5$.

Wpływ Normalizacji: Na wykresie wyraźnie widać, że krzywe dla danych **nieznormalizowanych** (niebieska i pomarańczowa) opadają wolniej i stabilizują się na wyższym poziomie błędów niż ich odpowiedniki dla danych znormalizowanych. Normalizacja (zarówno L1, jak i L2) znacząco przyspiesza zbieżność i pozwala osiągnąć niższy błąd końcowy. Dzieje się tak, ponieważ normalizacja skaluje dane do podobnego zakresu, co sprawia, że powierzchnia błędów staje się bardziej regularna i łatwiejsza do nawigacji dla algorytmu spadku gradientu.

Wpływ Funkcji Aktywacji: Porównując pary krzywych dla tej samej metody normalizacji (np. "Znormalizowane L2 + SIGMOID" vs "Znormalizowane L2 + RELU"), można zauważyć, że **ReLU** niemal w każdym przypadku prowadzi do szybszej zbieżności niż **Sigmoid**. Jest to spowodowane tym, że ReLU nie cierpi na problem zanikającego gradientu (ang. vanishing gradient) w takim stopniu jak Sigmoid, co ułatwia propagację błędów przez sieć i efektywniejszą aktualizację wag.

Najlepsza i najgorsza konfiguracja:

- Najszybszą zbieżność i najniższą stratę konsekwentnie osiągały konfiguracje wykorzystujące normalizację (L1 lub L2) oraz funkcję aktywacji ReLU.
- Najwolniejszą zbieżność obserwowano dla konfiguracji Nieznormalizowane + SIGMOID.

4. Wnioski Końcowe

Przeprowadzone eksperymenty pozwoliły sformułować następujące wnioski:

- Współczynnik uczenia jest kluczowym hiperparametrem. Jego odpowiedni dobór determinuje sukces procesu uczenia. Dla analizowanego problemu optymalne wartości LR leżały w przedziale od 0.75 do 1.0.

- Normalizacja danych wejściowych jest wysoce zalecana. Zarówno normalizacja L1, jak i L2, znacząco przyspieszyły zbieżność i pozwoliły na osiągnięcie niższej wartości funkcji straty w porównaniu do danych surowych.
- Funkcja aktywacji ReLU okazała się skuteczniejsza niż Sigmoid dla warstwy ukrytej w tym zadaniu, zapewniając szybszą naukę.
- Najlepsze rezultaty (najszybsza zbieżność do najniższego błędu) uzyskano dla konfiguracji "Znormalizowane L2 + RELU" przy współczynniku uczenia $LR = 1.0$. Ta kombinacja łączy zalety dobrze uwarunkowanego problemu (dzięki normalizacji) z efektywną propagacją gradientu (dzięki ReLU).